

PR-25

VERIFICATION NOTE

Executed reference-model audit

PUBLIC EVIDENCE SUPPLEMENT • v0.4 • 13 September 2026

STARGA

A finite logical model was checked. A physical robot was not.

Result

The original source package is internally intact: ZIP CRC checks pass, every listed SHA-256 digest matches, and the supplied checker regenerates its stored JSON byte for byte on CPython 3.13.5. The unmodified baseline has **3,078 reachable states and 83,106 event transitions**. Its original **33 selected faulty variants** are detected. The v0.4 verification adds a separately stated oracle and detects **42 selected variants in total**.

What changed

The supplied transition model has not been replaced to obtain a desired count. Its bytes are preserved. The new `pr25_verify.py` defines the expected conditions without calling the implementation's `normal_ok` helper, adds an ACTIVE-state consistency property, checks a separately written arithmetic transition specification, and supplies shared-helper regression mutations. A strict admission helper rejects malformed finite-code inputs without presenting itself as a production parser.

What this does not establish

This is not a controller implementation, physical safety proof, certification, commercial-system audit, or demonstration that the 25 requirement families are complete or independent. The model's trusted inputs and local atomic transitions leave physical and deployment obligations open. Separately expressing an oracle reduces one shared-code dependency; it does not create independent institutional review or eliminate common misunderstandings.

1. Reproduction and artifact identity

The following paths are relative to the public package:

reference_model/pr25_model_check.py	original, unchanged
reference_model/pr25_model_results.json	original, reproduced
reference_model/pr25_verify.py	v0.4 verification harness
reference_model/verified_results.json	executed v0.4 results
reference_model/ORIGINAL_README.md	historical source explanation

Run from reference_model/ using Python 3.10 or later, without optimization flags:

```
python3 pr25_model_check.py
python3 pr25_verify.py --output verified_results.json
```

The original command overwrites its adjacent original-results JSON; reproduce in a copy when preserving source bytes. The second command writes only its named output. Both use the standard library. The original module refuses `python -O`; the stronger harness uses explicit exceptions for its own checks and also imports that module. A nonzero process exit is a failed run, not an inconclusive PASS.

Reference source SHA-256

e5b69731fa56e4b0223a6c847c5a46cf23d834052d3b71403b37f3fb93311fe8

Original results SHA-256

49f6c73b7cd35f9d7e90be91d4b632edfe83d71718cb554f80d418e532b8d710

v0.4 verifier SHA-256

f886e034955b6143cf6c3673363a8293814f25bde523d1ac5d5a1ac1187ac740

The package manifest records all delivered-file digests. These are integrity checks, not signatures, certification, or evidence of the physical truth of inputs. The result includes the interpreter version; another supported interpreter version may change that metadata without changing the model counts.

2. Executed checks

Check	Result
Original baseline	3,078 reachable states; 83,106 transitions; 27 events/state
Original saved result	Byte-identical regeneration
Logical outputs reached	NONE, NORMAL, PROTECTIVE, NO_ASSURED_OUTPUT
Stronger baseline oracle	PASS over the complete reachable graph
Constructive reachability	Witness for each of 3,078 states; maximum witness length 12 events; closed under 83,106 transitions
Whole encoded transition relation	221,184 comparisons over 8,192 encodings x 27 events; zero mismatches
Admission accumulator	6 reachable states; 72 transitions
Bounded sequence equivalence	271,453 sequences; every length 0 through 5 over 12 letters
Selected mutation detection	27 verdict + 6 original supervisor + 9 shared-helper = 42/42
Malformed-input hardening	19 rejected examples

The 221,184 transition comparisons include encodings not reachable from the initial state. They are not 221,184 physical trials or an expansion of the 3,078-state reachable graph. The sequence enumeration is a bounded test; arbitrary-length fold equivalence additionally relies on the explicit induction in the paper. Case counts from different tests are not summed into a safety score.

3. A real verification weakness

In the supplied checker, both the transition implementation and the property checker call `normal_ok`. The audit changed only that shared helper to omit `required_set`, leaving the other eight normal prerequisites checked. The old checker returned PASS for the resulting faulty model: 3,088 states and 83,376 transitions. A direct witness also reaches a NORMAL grant with `required_set=False`.

This is a common-mode test-oracle weakness, not a counterexample to the original unmodified transition model. The repaired oracle spells out the nine conditions separately. It rejects the altered helper, including this shortest regression trace under the strengthened ACTIVE-state property:

```
issue
start
invalidate:required_set
```

The altered implementation leaves ACTIVE authority in place after a required condition is lost. The repaired check rejects that post-state. Analogous helper omissions for all nine prerequisites are detected. The exact traces appear in `verified_results.json`.

4. Counting and input-domain corrections

The original key `concrete_25_entry_cases: 302` is not a count of distinct 25-entry vectors. Its loop executes 300 positional cases, followed by one all-N/A 25-entry case and one empty-sequence case. Many positional cases repeat the all-PASS vector. There are 278 distinct sequences across those 302 executions. The corrected report records this breakdown without changing the original saved JSON.

The supplied admission function assumes its arguments are valid finite codes. Outside that domain, an unknown applicability code can be ignored and Python Boolean values can alias integer codes. These observations do not refute the declared finite-input model. The added strict helper rejects 19 malformed examples. It does not establish trusted applicability, authentic evidence, or that a caller supplied every required obligation ID.

Two original named supervisor mutants are coupled: the context mutant also removes a binding check, and the continuation mutant also leaves authority active during invalidation. Therefore 33 or 42 selected variants must not be marketed as independent faults, physical coverage, or a universal defect-detection percentage.

5. Model and assurance boundary

Ten Boolean summaries cover normal obligations, resolved applicability, a nonempty required set, approved profile, evidence, context, mission authority, enforcement health, recovery readiness, and logging. Four permit states and one freshness bit govern a single current permit. Each event is atomic at a local guard. Outputs are logical grants, not physical motion or elapsed control time.

The checked conditions require current normal prerequisites and a fresh binding for NORMAL; appropriate start/continue permit states; no normal replay; valid ACTIVE state; available protection not blocked by mission expiry or logging loss; and no claim of assured protection when readiness is absent. The model exposes `NO_ASSURED_OUTPUT` rather than inventing a fallback.

Sensor validity, physical dynamics and contact, real scheduling, actuator lag, stopping distance, protective feasibility, cryptographic correctness, persistent replay state, distributed owners, privacy enforcement, adaptation behavior, and deployment validation are outside this finite model. Faulty or bypassed real enforcement cannot be made reliable by assuming a Boolean health flag is true. No independent proof certificate is supplied.

Publication claim: Selected properties of the supplied finite admission model were exhaustively checked. The strengthened suite detected all 42 explicitly selected variants. This establishes neither all 25 physical-family predicates nor a verified robot implementation.